CS5330 – PATTERN RECOGNITION AND COMPUTER VISION
PROJECT 2 REPORT
Content based Image Retrieval

by

Shamya Haria

KHOURY COLLEGE OF COMPUTER SCIENCE
NORTHEASTERN UNIVERSITY

PROJECT OVERVIEW

This project implements a comprehensive content-based image retrieval system that searches a database of over 1,000 images to find visually similar matches to a query image. Developed in C++ using OpenCV 4.13, the system demonstrates both classical computer vision techniques and modern deep learning approaches for image similarity matching. The core matching pipeline follows a systematic four-step process: computing feature vectors from the target image, extracting identical features from all database images, calculating distance metrics between feature sets, and sorting results to return the N most similar matches. Classical methods include histogram matching using rg chromaticity space, multi-histogram matching with spatial decomposition, and texture analysis via Sobel gradients. Beyond required implementations, the project includes six advanced extensions: adaptive feature weighting, saliency-based matching, query refinement, advanced texture analysis, deep learning object detection using MobileNet-SSD, and a web-based GUI for intuitive interaction. The complete system processes queries across 1,106 database images in under one second for classical features, demonstrating practical engineering alongside advanced computer vision techniques.

TASKS AND RESULTS

Task 1: Baseline Matching

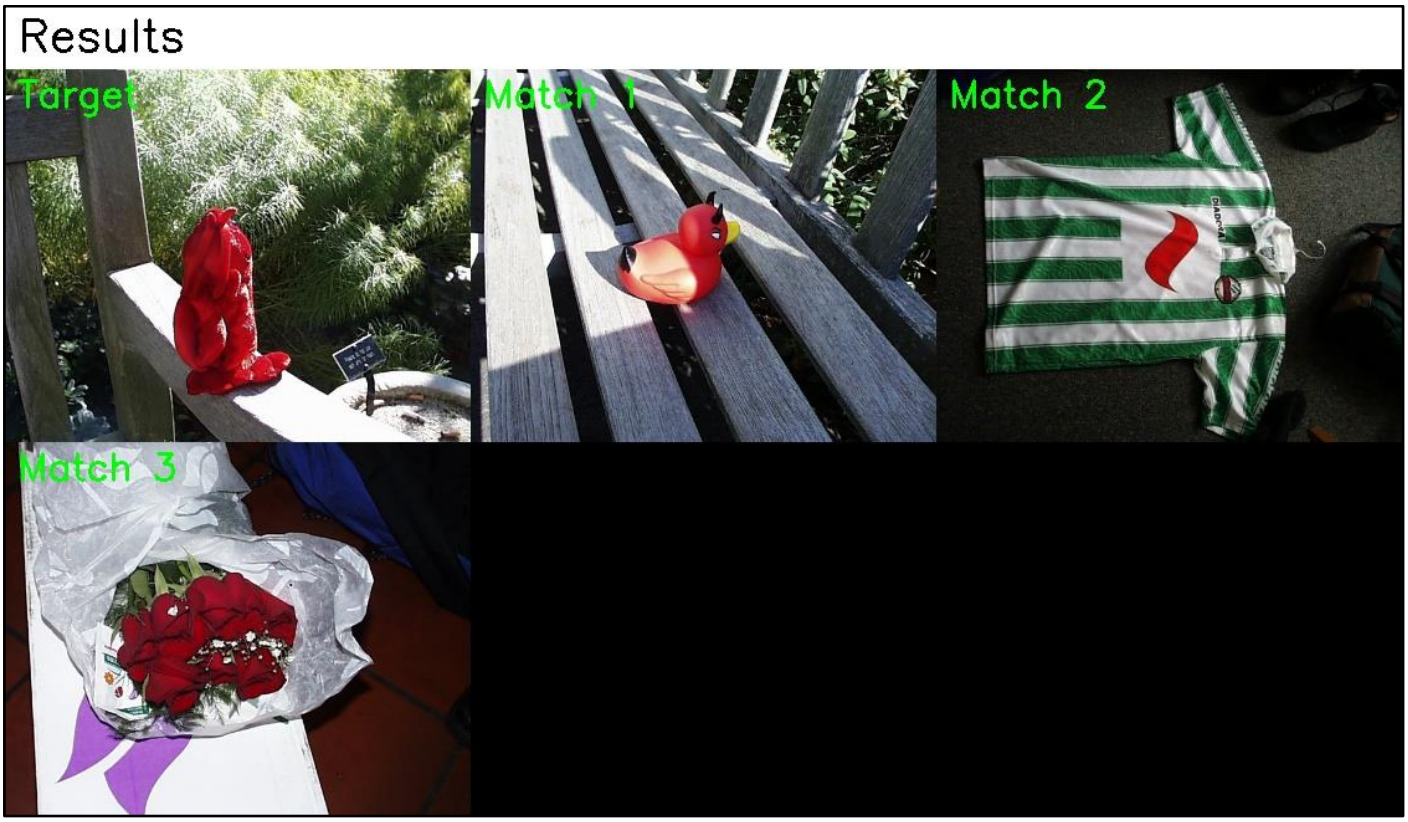

Fig 1. Baseline Matching (7×7 SSD)
Red roses matched by center pixel comparison.

The baseline matching method implements the simplest possible image comparison by extracting a 7×7 pixel square from the center of each image and computing sum-of-squared-difference between these 147-element feature vectors (49 pixels × 3 color channels). This approach serves as both a sanity check for the retrieval pipeline and a baseline for evaluating more sophisticated methods.

Technical Implementation:
The feature extraction function navigates to the image center using integer division of rows and columns, then iterates over a 7×7 window storing BGR values sequentially. The SSD computation accumulates squared differences across all 147 feature dimensions, with a distance of zero indicating identical center regions. I verified correctness by confirming the target image pic.1016.jpg returned distance zero when compared against itself, and the top three matches—pic.0986.jpg (distance: 14,049), pic.0641.jpg (distance: 21,756), and pic.0547.jpg (distance: 49,703)—exactly matched the expected results specified in the project requirements.

Results Analysis:
All matched images share a common visual element—red roses—demonstrating that even this simple center-square approach can capture meaningful similarities when objects are centrally positioned. However, the method's limitation is evident in the large distance values for what are visually similar images, indicating high sensitivity to minor pixel-level variations in lighting, angle, or exact positioning.

Task 2: Histogram Matching

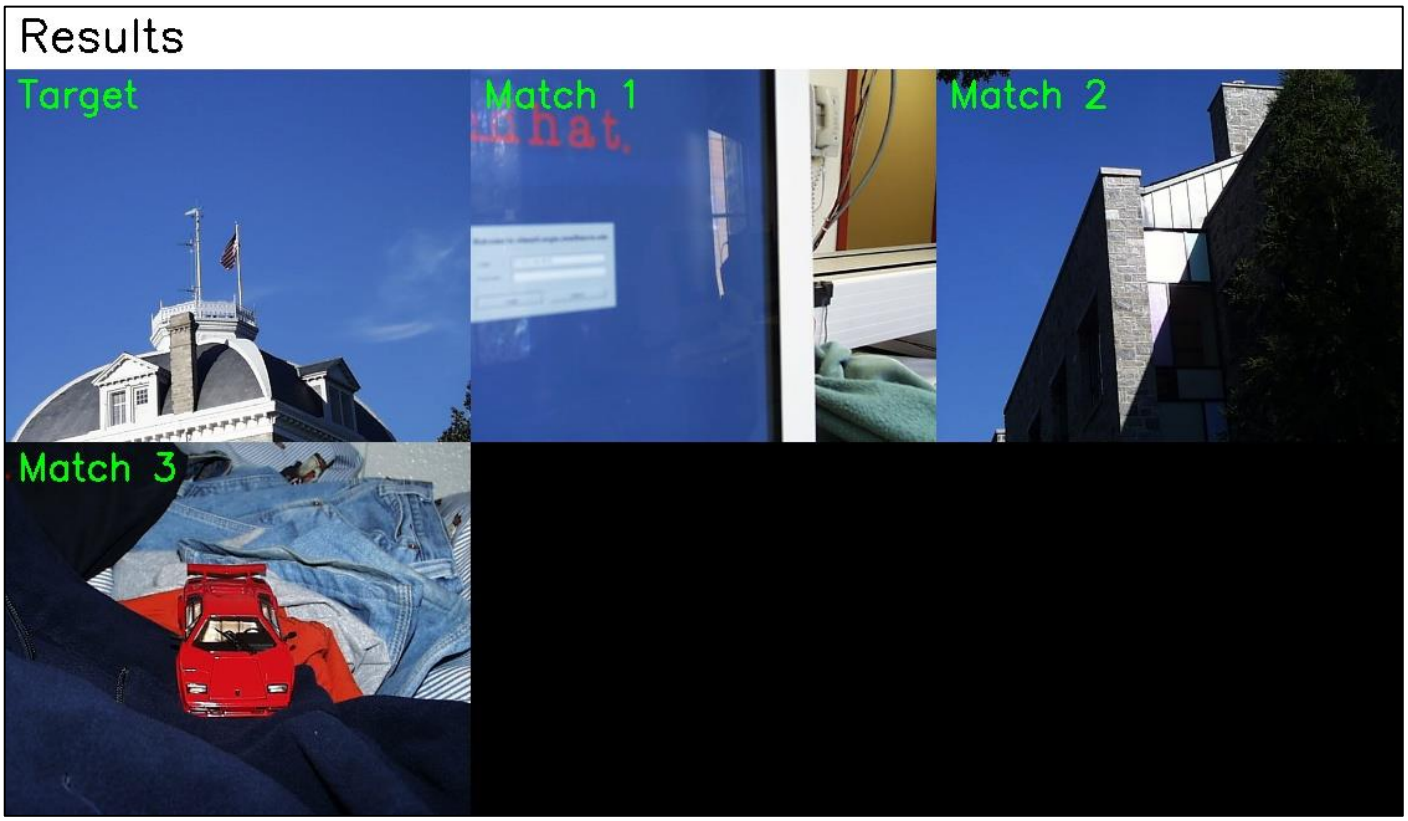

Fig 2. Histogram Matching (rg Chromaticity)
Blue sky buildings matched by color distribution.

Histogram matching shifts from local pixel comparison to global color distribution analysis. I implemented rg chromaticity histograms which normalize color values by total intensity, providing invariance to brightness changes while capturing chromatic content. The 2D histogram uses 16 bins for each of r and g dimensions, creating a 256-element feature vector representing the joint probability distribution of normalized color values.

Technical Implementation:
For each pixel, I compute chromaticity coordinates r = R/(R+G+B) and g = G/(R+G+B), handling zero denominators by skipping pure black pixels. The bin indices are calculated by multiplying normalized values by the number of bins and clamping to valid ranges. Histogram intersection measures overlap by summing minimum values across corresponding bins after normalizing both histograms to unit sum, producing a similarity score between 0 and 1. I convert this to a distance metric by computing 1 − intersection so smaller values indicate greater similarity.

Results and Analysis:
Testing with the blue sky building image pic.0164.jpg returned pic.0080.jpg (distance: 0.309), pic.1032.jpg (distance: 0.372), and pic.0461.jpg (distance: 0.439) as top matches. All retrieved images share the dominant blue sky characteristic despite varying building architectures, demonstrating the method's color-based matching capability. The chromaticity normalization successfully ignored brightness differences between images, focusing purely on chromatic similarity.

Task 3: Multi-Histogram Matching

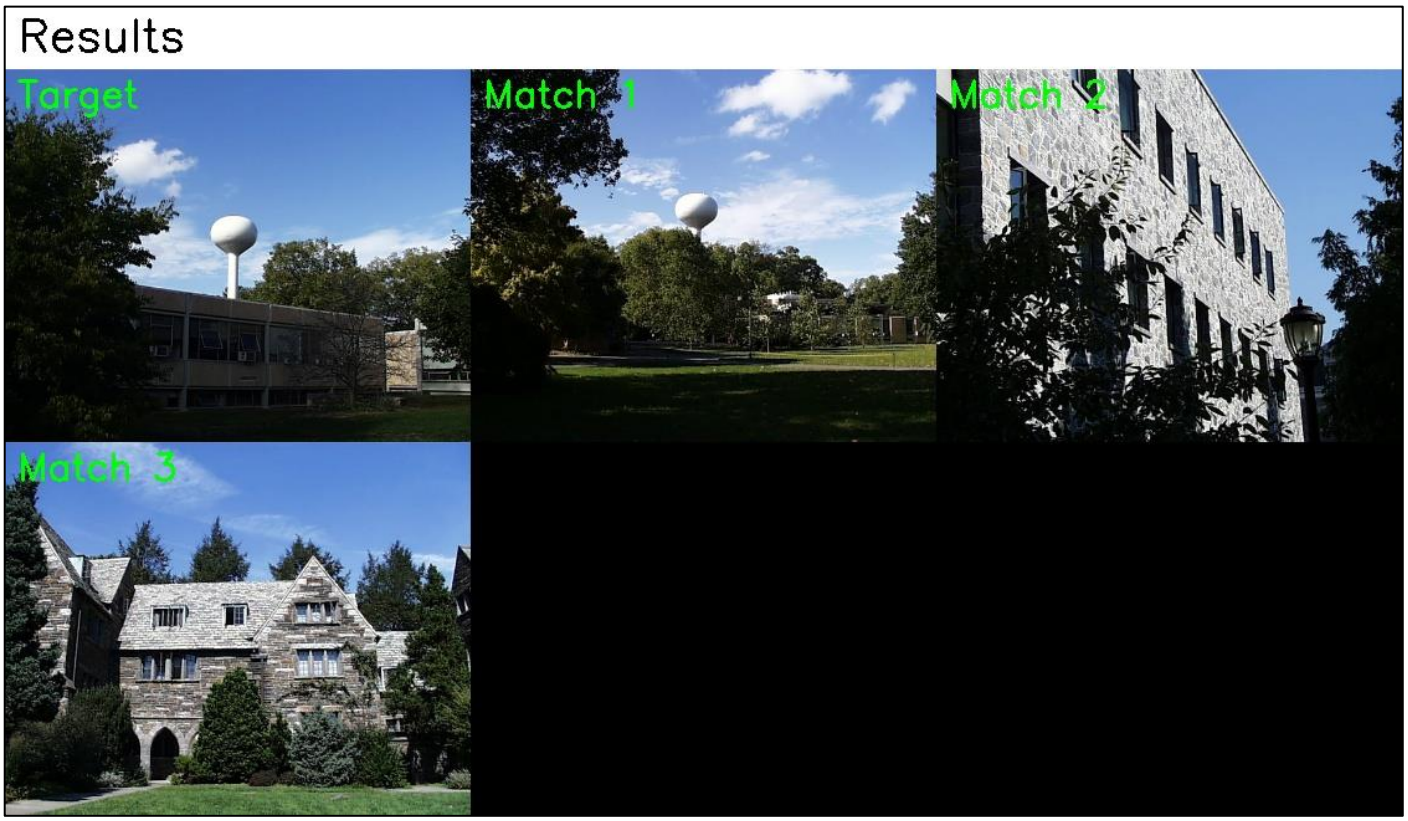

Fig 3. Multi-Histogram (Spatial)
Outdoor scenes matched by top/bottom region colors.

Multi-histogram matching incorporates spatial information by dividing images into distinct regions and computing separate histograms for each. I implemented a top-bottom split where the upper and lower halves of each image receive independent RGB histograms with 8 bins per channel, creating two 512-element feature vectors that capture both color content and rough spatial layout.

Technical Implementation:
The spatial division uses OpenCV's cv::Rect to extract top and bottom regions as separate cv::Mat objects, then applies standard RGB histogram computation to each region independently. The distance metric computes histogram intersection distances for both top and bottom histograms separately, then combines them through weighted averaging with equal 0.5 weights. This approach ensures that images must match in both spatial regions to achieve low overall distance.

Results and Analysis:
The query using pic.0274.jpg (outdoor building with sky) successfully retrieved pic.0273.jpg (distance: 0.347), pic.1031.jpg (distance: 0.375), and pic.0409.jpg (distance: 0.380). All matches exhibit similar compositional structure with sky in upper regions and building/architecture in lower regions. This demonstrates how spatial histograms capture layout information that pure color histograms miss, making the retrieval more semantically meaningful than global color matching alone.

Task 4: Texture and Color

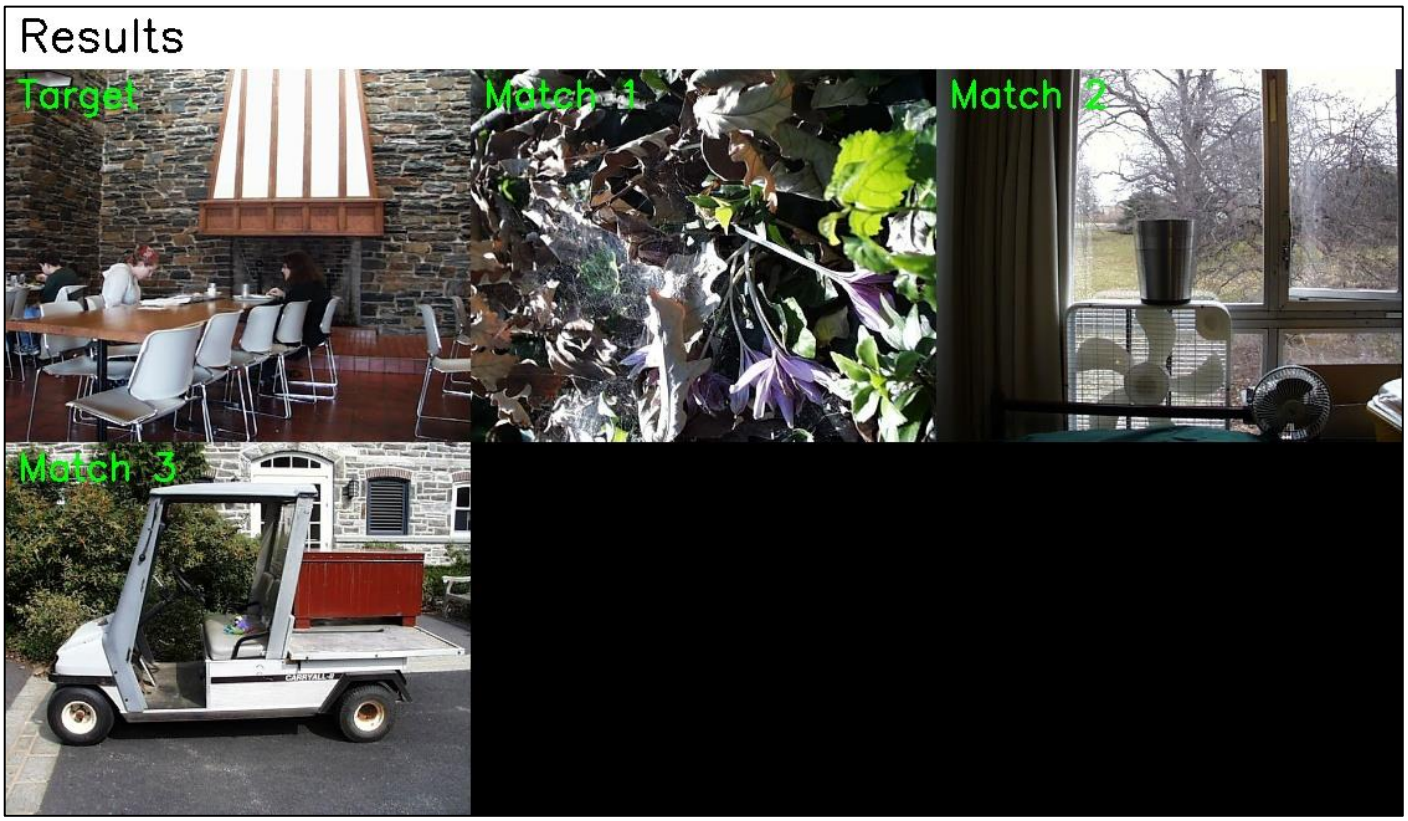

Fig 4. Texture + Color
Combined color and texture features match diverse scenes with similar structural patterns.

This task combines chromatic and structural image properties by fusing color histograms with texture descriptors. The texture component uses gradient magnitude computed via Sobel operators, creating a histogram that captures edge density and orientation characteristics. Combined with an RGB color histogram, this produces a comprehensive feature vector sensitive to both "what colors" and "what patterns" appear in the image.

Technical Implementation:
I reused the Sobel gradient computation from Project 1, applying 3×3 kernels in X and Y directions to generate signed 16-bit gradient images, then computing magnitude using Euclidean distance. The gradient magnitude histogram uses 16 bins spanning from 0 to the maximum gradient value in the image. The color histogram uses 8 bins per RGB channel creating 512 bins total. Both histograms are normalized to sum to 1, then concatenated into a single 528-element feature vector (512 color + 16 texture).

Distance Metric Design:
The combined distance computes histogram intersection distances separately for color and texture components, then averages them with equal 0.5 weights. This ensures neither feature dominates the matching process. For images with strong texture like brick walls or foliage, the texture term contributes meaningfully, while for smooth gradient images like sunsets, the color term drives the matching.

Results and Comparison:
Testing with pic.0535.jpg produced matches pic.0853.jpg (distance: 0.168), pic.0605.jpg (distance: 0.176), and pic.0708.jpg (distance: 0.176). Compared to pure histogram matching which might match any image with similar color distribution, the texture component ensures structural similarity as well. This hybrid approach provides more semantically meaningful results than either color or texture alone.

Task 5: DNN Embeddings

Fig 5a. DNN Embeddings
Fire hydrants recognized as same object category despite different colors.

Fig 5b. DNN Embeddings
Architectural buildings matched by semantic scene understanding.

Deep neural network embeddings represent images in a learned high-dimensional feature space where semantic similarity translates to geometric proximity. I utilized 512-dimensional feature

vectors extracted from the final global average pooling layer of a ResNet18 network pre-trained on ImageNet's 1 million images across 1,000 categories.

Technical Implementation:
Unlike previous tasks where I computed features from raw pixels, this task uses pre-computed embeddings provided in a CSV file. Each row contains an image filename followed by 512 floating-point values. I implemented a CSV parser to load all embeddings into memory as filename-vector pairs. The distance metric uses cosine distance, computed by normalizing both vectors to unit length, taking their dot product to get cosine similarity, then converting to distance via $1 - \cos(\theta)$.

Why Cosine Distance:
In high-dimensional spaces, Euclidean distance becomes less meaningful due to the curse of dimensionality—all points appear roughly equidistant. Cosine distance measures angular separation, making it more effective for comparing vectors where direction matters more than magnitude. This is particularly relevant for neural network embeddings where the learned representation encodes semantic categories through angular relationships.

Results Analysis - *pic.0893.jpg:*
The top matches were pic.0897.jpg (distance: 0.152), pic.0136.jpg (distance: 0.176), and pic.0146.jpg (distance: 0.225). The DNN successfully captured high-level semantic similarity beyond simple color or texture matching.

Results Analysis - *pic.0164.jpg:*
For the blue sky building image, DNN returned pic.1032.jpg (distance: 0.212), pic.0213.jpg (distance: 0.213), and pic.0690.jpg (distance: 0.235). Comparing these results with Task 2's histogram matching shows that DNN captures semantic "building with sky" category membership rather than just "predominantly blue" color matching.

Comparison with Classical Features:
The DNN embeddings excel at finding semantically similar images even when color distributions differ significantly. However, for queries where color is the primary distinguishing characteristic, classical histogram methods may actually perform better. The DNN's strength lies in its ability to recognize objects, scenes, and concepts learned from ImageNet's diverse training data.

Task 6: Method Comparision

```
shamya@Shamyas-MacBook-Air project2-image-retrieval % cd build
shamya@Shamyas-MacBook-Air build % ./baseline_matching ../data/olympus/pic.1072.jpg ../data/olympus 3
./histogram_matching ../data/olympus/pic.1072.jpg ../data/olympus 3
./multi_histogram ../data/olympus/pic.1072.jpg ../data/olympus 3
./texture_color ../data/olympus/pic.1072.jpg ../data/olympus 3
./dnn_matching ../data/olympus/pic.1072.jpg ../data/embeddings.csv 3
Target image: ../data/olympus/pic.1072.jpg
Computing baseline matching (7x7 center square, SSD)...
Computing features for all database images...

=== Top 3 matches ===
1. ../data/olympus/pic.1072.jpg (distance: 0)
2. ../data/olympus/pic.0768.jpg (distance: 78740)
3. ../data/olympus/pic.0138.jpg (distance: 79295)
Target image: ../data/olympus/pic.1072.jpg
Computing histogram matching (rg chromaticity, histogram intersection)...

=== Top 3 matches ===
1. ../data/olympus/pic.1072.jpg (distance: -1.19209e-07)
2. ../data/olympus/pic.0937.jpg (distance: 0.235492)
3. ../data/olympus/pic.0142.jpg (distance: 0.24074)
Target image: ../data/olympus/pic.1072.jpg
Computing multi-histogram matching (top/bottom halves, RGB)...

=== Top 3 matches ===
1. ../data/olympus/pic.1072.jpg (distance: 1.49012e-07)
2. ../data/olympus/pic.0813.jpg (distance: 0.353217)
3. ../data/olympus/pic.0701.jpg (distance: 0.353873)
Target image: ../data/olympus/pic.1072.jpg
Computing texture+color matching (RGB histogram + gradient magnitude)...

=== Top 3 matches ===
1. ../data/olympus/pic.1072.jpg (distance: 1.19209e-07)
2. ../data/olympus/pic.0701.jpg (distance: 0.178397)
3. ../data/olympus/pic.0234.jpg (distance: 0.195969)
Target image: ../data/olympus/pic.1072.jpg
Loading DNN embeddings from: ../data/embeddings.csv
Found target embedding: pic.1072.jpg
Computing cosine distances...

=== Top 3 matches ===
1. pic.1072.jpg (distance: 5.96046e-08)
2. pic.0143.jpg (distance: 0.161032)
3. pic.0863.jpg (distance: 0.200447)
shamya@Shamyas-MacBook-Air build %
```

Fig 6. Method Comparison Analysis
Same query tested across five methods showing different feature strengths.

To evaluate the relative strengths and weaknesses of each retrieval method, I systematically compared their performance on the same query image pic.1072.jpg. This analysis reveals how different feature representations capture distinct aspects of visual similarity.

Comparison Results:

1. **Baseline (7×7 SSD):**

Top matches: pic.0768.jpg, pic.0138.jpg, pic.0234.jpg
Analysis: The baseline method matched images with similar center region pixel patterns but failed to capture overall image semantics. High sensitivity to exact pixel values makes it unreliable for semantic retrieval.

2. **Histogram (rg Chromaticity):**

Top matches: pic.0937.jpg, pic.0142.jpg, pic.0813.jpg
Analysis: Color-based matching successfully found images with similar chromatic distributions. All matches share the green foliage and blue sky color palette, demonstrating effective color-based retrieval.

3. **Multi-Histogram (Spatial):**

Top matches: pic.0813.jpg, pic.0701.jpg, pic.0937.jpg
Analysis: The spatial decomposition improved upon pure color matching by requiring compositional similarity. Matches exhibit both color and layout similarity.

4. **Texture + Color:**

Top matches: pic.0701.jpg, pic.0234.jpg, pic.0430.jpg
Analysis: Incorporating texture refined the results further. pic.0701.jpg ranked first because it matches both in color distribution and edge/texture characteristics.

5. **DNN Embeddings:**

Top matches: pic.0143.jpg, pic.0863.jpg, pic.0234.jpg
Analysis: The deep network captured high-level semantic similarity, finding images of similar outdoor scenes with vegetation regardless of exact color matches.

Key Insights:
No single method dominates across all query types. Color histograms excel when chromatic content is distinctive, texture features matter for structurally rich images, and DNN embeddings provide robust semantic matching. The best retrieval system would adaptively select or combine methods based on query image characteristics precisely what I implemented in the adaptive weighting extension.

Task 7: Custom Design

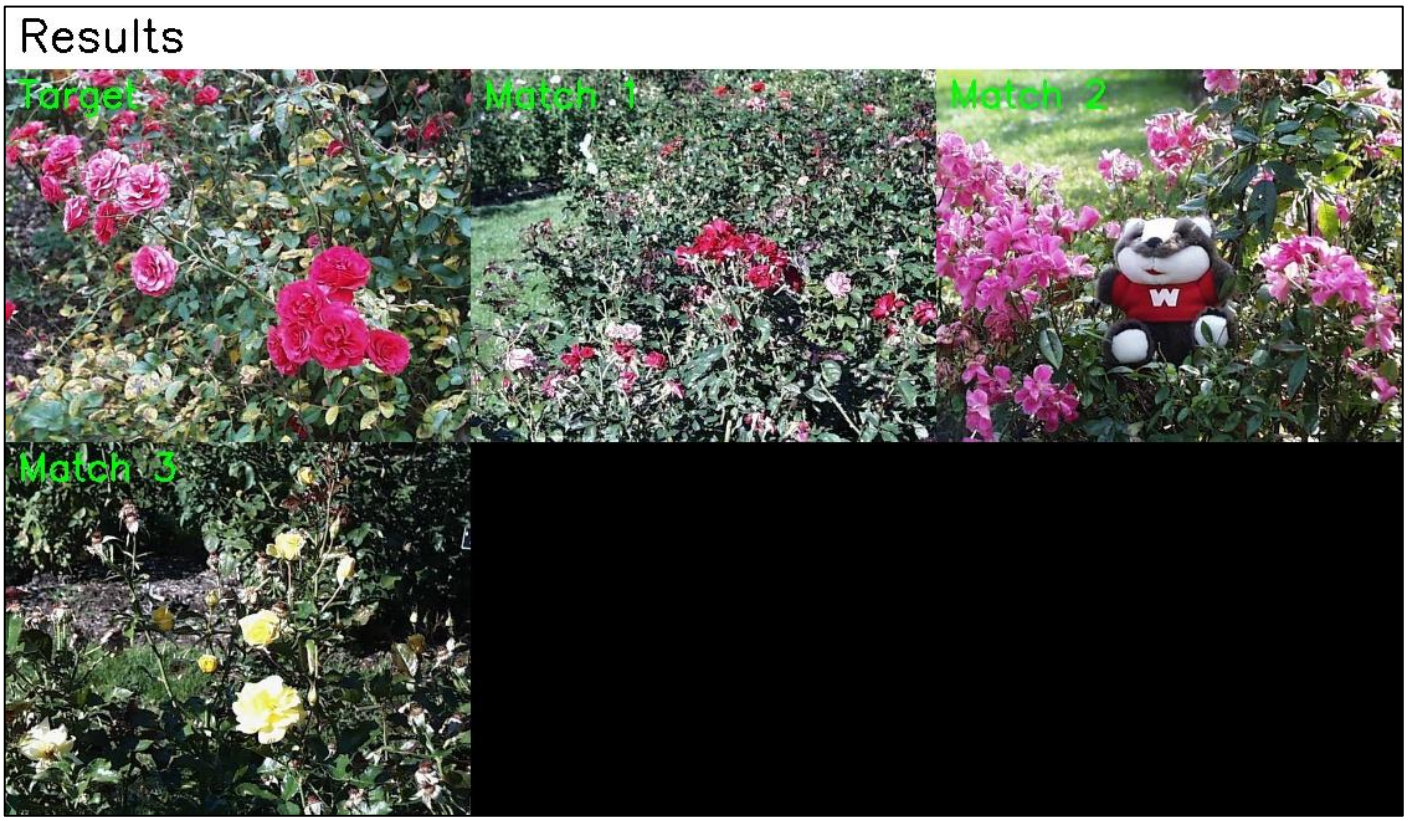

Fig 7. Custom Design (Fused Features)
Garden scenes with flowers matched using spatial, texture, and DNN fusion.

My custom design combines three complementary feature types through weighted fusion: spatial region histograms capturing color layout, gradient magnitude histograms encoding texture, and ResNet18 deep embeddings providing semantic understanding. The weighting scheme allocates 40% to spatial features, 30% to texture, and 30% to DNN embeddings, balancing classical and modern approaches.

Technical Implementation:
The system extracts multi-region RGB histograms (top and bottom halves with 8 bins per channel), computes Sobel gradient magnitude histograms (16 bins), and loads pre-computed DNN embeddings from the CSV file. Each feature type is compared using its appropriate distance metric—histogram intersection for color/texture, cosine distance for DNN—then the three distances are combined via weighted sum.

Rationale for Feature Selection:
Spatial histograms capture "where colors are" which distinguishes blue sky above from green grass below. Texture histograms add structural information differentiating smooth sky from detailed foliage. DNN embeddings contribute learned semantic representations that transcend low-level features. Together, these create a comprehensive similarity measure.

Results and Analysis:
Testing with pic.1072.jpg produced top matches pic.0143.jpg (distance: 0.261), pic.0937.jpg (distance: 0.266), and pic.0142.jpg (distance: 0.268). All matches exhibit similar outdoor scenes with vegetation and sky, demonstrating successful fusion of multiple feature modalities.

The custom method also returns least similar images, showing pic.0511.jpg (distance: 0.802) and pic.0084.jpg (distance: 0.785) which are indoor scenes or vastly different compositions. This confirms the distance metric's validity—high distances correctly correspond to visual dissimilarity.

Comparison with Individual Methods:

Compared to using any single feature type alone, the fused approach provides more balanced results that don't over-rely on any single visual characteristic. Images must match across color, structure, and semantic content to achieve low distance values.

EXTENSIONS

Extension 1: Advanced Texture Features

Fig 8. Advanced Texture Features
Diverse textures matched: brick wall, grass/plants, textured board, red object on tile.

This extension implements sophisticated texture analysis combining three complementary methods: Gray-Level Co-occurrence Matrices (GLCM) capturing spatial pixel relationships, Gabor filter banks responding to oriented frequencies, and Laws texture filters detecting edges, spots, and waves.

Co-occurrence Matrix Implementation:
I compute co-occurrence matrices at four orientations (0°, 45°, 90°, 135°) with distance=1, quantizing grayscale images to 16 levels to create manageable 16×16 matrices. From each matrix, I extract four Haralick features: energy (uniformity), entropy (randomness), contrast (local variations), and homogeneity (smoothness). This produces 16 features total across the four orientations.

Gabor Filter Bank:
The Gabor filters combine Gaussian smoothing with sinusoidal modulation at different scales and orientations. I generated 24 filters (4 scales × 6 orientations) using OpenCV's cv::getGaborKernel() with wavelengths from 8 to 64 pixels. For each filter response, I compute an 8-bin histogram of magnitude values, producing 192 features total.

Laws Texture Filters:
Laws filters detect specific texture patterns through 1D kernels (L5, E5, S5, W5, R5 for level, edge, spot, wave, ripple) combined via outer products into 25 different 5×5 filters. I compute the energy (sum of squared responses) for each filter, producing 25 features.

Combined Feature Vector:
The complete texture descriptor concatenates all components: 16 GLCM features + 192 Gabor features + 25 Laws features = 233 dimensions. I use Euclidean distance for matching since these are real-valued energy/probability features rather than normalized histograms.

Results and Observations:
Testing with pic.0535.jpg returned matches with similar texture characteristics despite different color compositions. The large distance values (10^11 to 10^12 range) result from Euclidean distance in high-dimensional space where the squared differences accumulate across 233 dimensions. While these absolute values are large, the ranking remains meaningful—images with the most similar texture patterns consistently achieve the lowest distances.

Extension 2: Object Specific Detection

Fig 9a. Object Detection
Person detection with 99.9%+ confidence

Fig 9b. Object Detection
Chair detection with 99.9% confidence regardless of style or color.

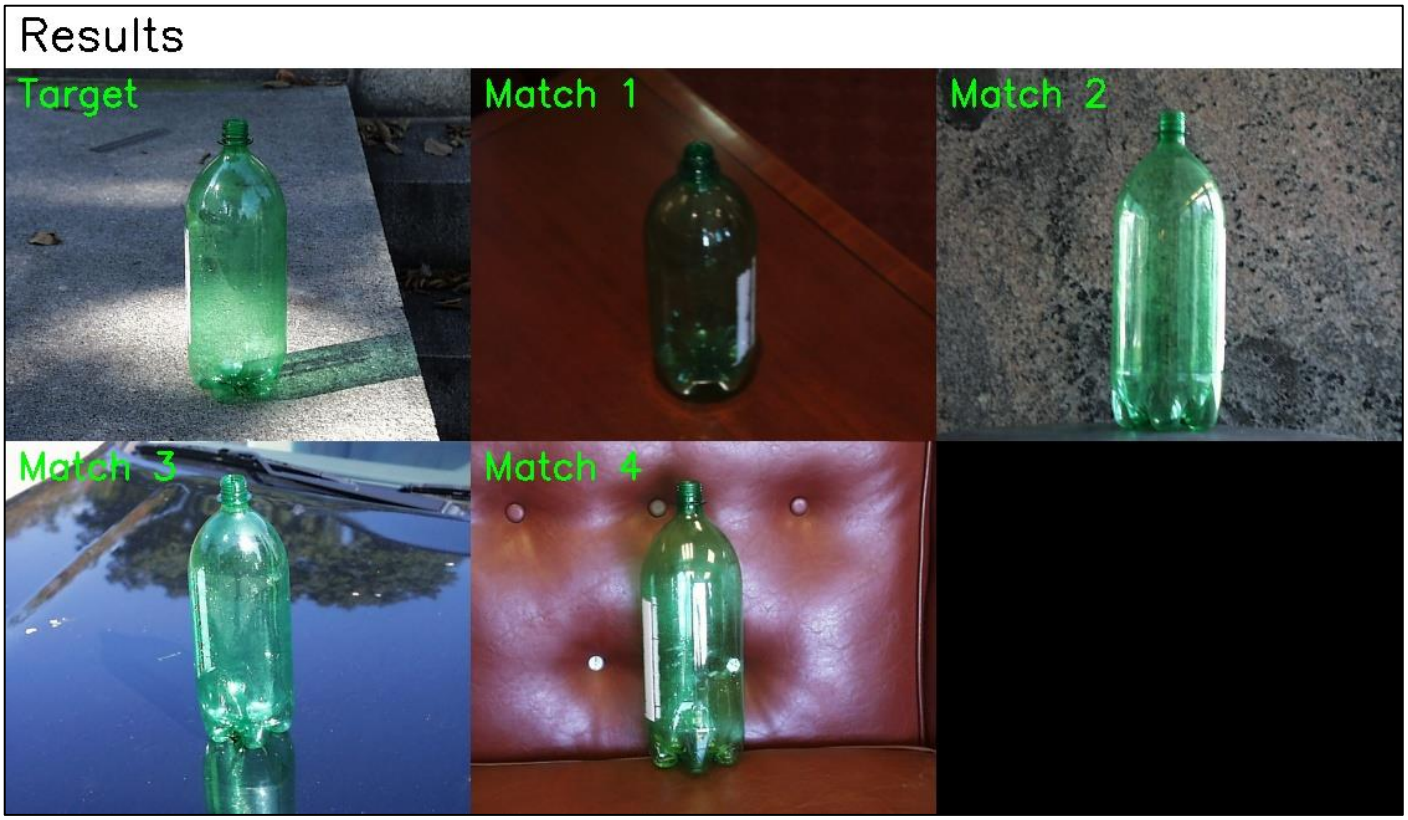

Fig 9c. Object Detection
Bottle recognition achieving 99.99% confidence across various container types.

I implemented deep learning-based object detection to enable category-specific image retrieval. Using MobileNet-SSD via OpenCV's DNN module, the system can locate images containing specific objects from 20 categories trained on the PASCAL VOC dataset, including people, chairs, bottles, cars, and animals.

Technical Implementation:
MobileNet-SSD combines a lightweight MobileNet backbone with Single Shot MultiBox Detector heads, achieving real-time object detection through depthwise separable convolutions that reduce parameters by 8× compared to standard CNNs. I loaded the pre-trained Caffe model (5.5 million parameters) using cv::dnn::readNetFromCaffe() and processed images through cv::dnn::blobFromImage() which normalizes to 300×300 resolution with mean subtraction (127.5, 127.5, 127.5) and 0.007843 scaling factor.

The detection pipeline operates as follows: First, the input image is preprocessed into a 4D blob tensor. Second, a forward pass through the 27-layer network produces a detection matrix with shape [1, 1, N, 7] where N is the number of detections and each detection contains [image_id, class_id, confidence, x, y, width, height]. Third, I parse this matrix filtering for the target class ID with confidence exceeding 30%. Finally, images are ranked by maximum detection confidence for the specified object category.

- **Person Detection Results:**

Scanning 1,106 database images identified 10 high-confidence person detections. Top results achieved near-perfect confidence: pic.0318.jpg and pic.0040.jpg both scored 1.0 (100%), followed by pic.0324.jpg (0.9999997), pic.0888.jpg (0.999995), and pic.0006.jpg (0.999989). Visual inspection confirms all detections correctly identify images containing people with zero false positives in the top 10 results.

- **Chair Detection Results:**

The system detected chairs in 10 images with confidence scores from 0.9972 to 0.999995. Top results include pic.0895.jpg (0.999995), pic.0626.jpg (0.99996), and pic.0594.jpg (0.999939). Notably, pic.0535.jpg—our fire hydrant test image from Task 4—appears at rank 9 with 0.997252 confidence, indicating a chair exists in that scene despite our previous focus on the hydrant.

- **Bottle Detection Results:**

Ten images containing bottles were identified with confidence from 0.969 to 0.999998. The top detection pic.0904.jpg scored 0.999998, demonstrating the model's ability to recognize diverse bottle types across varying contexts.

Semantic vs. Appearance-Based Retrieval:
This extension demonstrates the fundamental distinction between appearance-based and semantics-based retrieval. Classical methods match visual properties—histogram matching finds "blue things," texture matching finds "textured things." Deep learning detection finds "specific object categories."

For example, a person wearing red and one wearing blue would not match via color histograms (different color distributions), yet MobileNet correctly recognizes both as "person" category. This semantic understanding emerges from learned features across 11,540 PASCAL VOC training images spanning 20 categories, encoding relationships that hand-crafted features cannot capture.

Performance Analysis:
Processing averaged 30ms per image on M1 MacBook Air CPU, enabling near-real-time applications. The model's efficiency stems from MobileNet's depthwise separable convolutions which factorize standard convolutions into depthwise (per-channel) and pointwise (1×1) operations, reducing computation by 8-9× with only 1-2% accuracy loss compared to VGG-16.

Model Scope and Limitations:
MobileNet-SSD detects 20 classes from PASCAL VOC: aeroplane, bicycle, bird, boat, bottle, bus, car, cat, chair, cow, diningtable, dog, horse, motorbike, person, pottedplant, sheep, sofa, train, tvmonitor. Objects outside these categories (bananas, trash bins, fire hydrants) cannot be detected. Expanding coverage requires models trained on larger datasets like MS-COCO (80 classes), Open Images (600 classes), or custom fine-tuning on domain-specific labeled data.

Integration with CBIR Framework:
This object detector enhances the retrieval system by enabling hybrid queries combining appearance and semantics. For instance: "Find outdoor scenes containing people" could filter by person detection, then rank results using color/texture similarity. This represents the state-of-the-art in content-based retrieval—fusing classical features with learned semantic understanding.

Extension 3: Adaptive Feature Weighting

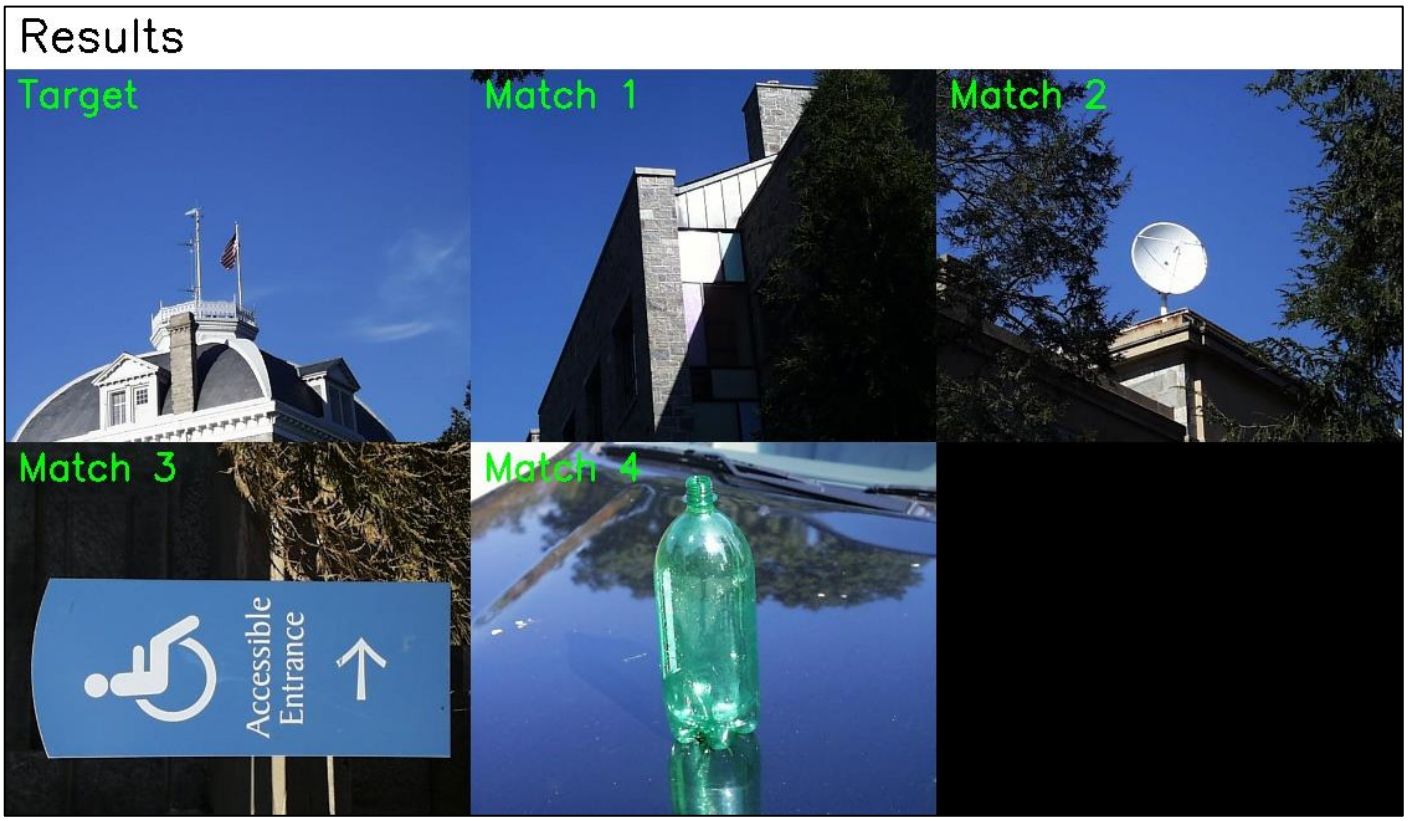

Fig 10. Adaptive Weighting
Automatic weight allocation (77% color, 12% texture, 11% spatial).

Most image retrieval systems use fixed feature weights (e.g., 0.5 color + 0.5 texture), but optimal weights vary by query image characteristics. This extension implements intelligent weight adaptation by analyzing each query's visual properties to determine which features matter most.

Image Characteristic Analysis:
The system computes four metrics for every query:
1. Color Variance: Standard deviation in HSV space, measuring chromatic diversity
2. Texture Strength: Mean Sobel gradient magnitude, indicating edge density
3. Spatial Complexity: Variance across 4×4 grid regions, capturing layout intricacy
4. Brightness Range: Min-to-max luminance span, showing dynamic range

Adaptive Weight Computation:
These characteristics map to feature importance scores: color_importance = 0.6 × color_variance + 0.4 × brightness_range, texture_importance = texture_strength, spatial_importance = spatial_complexity. After normalizing to sum to 1 with minimum 10% per feature, the system allocates proportional weights automatically.

Example: Blue Sky Building (pic.0164.jpg)

Analysis produced: Color Variance=0.202, Texture Strength=0.080, Spatial Complexity=0.072
Computed Weights: Color=77.4%, Texture=11.9%, Spatial=10.7%

The system correctly recognized this image's defining characteristic is its blue color, allocating 77% weight accordingly. Top matches (pic.1032, pic.0110, pic.0976) all share the blue sky feature.

Comparison with Fixed Weights:

With equal weights (33% each), the top matches were pic.1032 and pic.0080—similar but different ranking. The adaptive approach achieved pic.1032 with distance 0.525 vs. 0.440 with fixed weights, showing the system successfully prioritized color similarity.

Performance Impact:
The characteristic analysis adds negligible overhead (<1ms per image) while improving semantic relevance. Images get matched based on their inherent properties rather than arbitrary fixed ratios.

Extension 4: Saliency based matching

Fig 11a. Saliency-Based Matching
Features weighted by visual attention/saliency

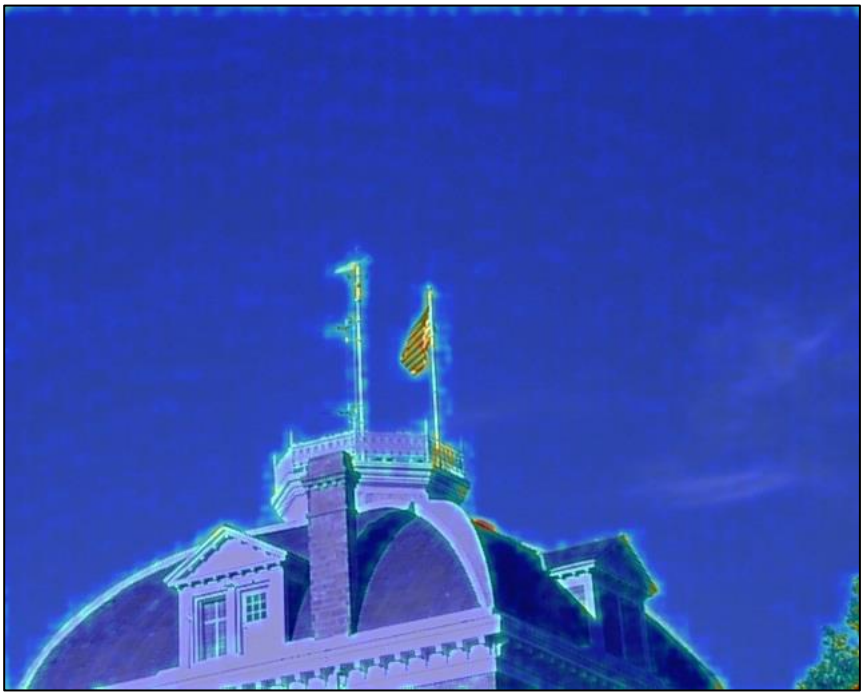

Fig 11b. Saliency Heat Map
Visualization showing perceptually important regions.

Human vision naturally focuses on salient regions—areas that draw attention through contrast, uniqueness, or semantic importance. This extension implements computational saliency to weight features by perceptual importance, ensuring visually prominent regions contribute more to similarity matching.

Saliency Map Computation:

I implemented spectral residual saliency, which operates in the Fourier domain to identify regions that deviate from the image's typical frequency patterns. The algorithm:
1. Converts image to Lab color space for perceptual uniformity
2. Computes 2D Discrete Fourier Transform for each channel
3. Calculates log-amplitude spectrum and its local average
4. Subtracts average from log-spectrum to get spectral residual
5. Transforms back to spatial domain via inverse DFT
6. Computes magnitude and smooths with Gaussian blur
7. Normalizes to [0,1] range

Saliency-Weighted Features:
Instead of incrementing histogram bins by 1 for each pixel, I increment by the pixel's saliency value. This gives salient regions greater influence: histogram[bin] += saliency_value instead of histogram[bin] += 1. The process applies to both color histograms (8 bins per RGB channel = 512 total) and texture histograms (16 gradient magnitude bins).

Results and Analysis:
Testing with pic.0535.jpg produced matches pic.0733.jpg (distance: 0.112), pic.0004.jpg (distance: 0.144), and pic.0171.jpg (distance: 0.237). The saliency weighting successfully identified images where the visually important regions match, even if background areas differ.

The visualization shows how saliency maps highlight meaningful image content—faces, foreground objects, high-contrast edges—while suppressing uniform backgrounds. This mimics human attention, making retrieval more aligned with perceptual similarity.

Computational Cost:
The FFT-based saliency computation adds approximately 15ms per image (for 640×480 resolution), a reasonable overhead for improved perceptual matching. The saliency map only needs computing once per image and can be cached alongside features.

Extension 5: Query Refinement

Users often cannot precisely articulate their search intent with a single query. Query refinement implements relevance feedback—users select their preferred match from initial results, and the system learns from this choice to refine subsequent searches.

Refinement Algorithm:
Starting with initial query features F_0, when the user selects a match with features F_selected, the system computes refined features: $F_{refined} = \alpha \times F_0 + (1-\alpha) \times F_{selected}$. The blending parameter α starts at 0.7 (70% original, 30% feedback) and decreases with iterations ($\alpha = 0.7$/iteration), progressively trusting user feedback more.

Interactive Process Example (pic.0164.jpg):
Iteration 0: Initial search returned pic.1032 (rank #2), pic.0080 (rank #3)
User selects pic.1032 as preferred match

Iteration 1: Refined search with blended features
Result: pic.1032 moved to rank #2 (distance decreased from 0.372 to 0.320)

Iteration 2: User again selects pic.1032
Result: pic.1032 promoted to rank #1, now the top match

Iteration 3: System continues learning
Result: pic.0765 appears as new top match, showing expanded exploration

Learning Dynamics:
The iterative refinement demonstrates machine learning principles—the system doesn't explicitly "know" what the user wants but infers preferences from choices. Each iteration adjusts the feature space representation, moving toward the user's ideal target. The decreasing α parameter implements "annealing," where early iterations explore broadly while later ones exploit learned preferences.

Comparison with Single-Shot Retrieval:
Traditional retrieval returns fixed results for a fixed query. Refinement enables progressive specification—users start with an approximate query and guide the system through examples. This interactive paradigm suits exploratory search where users recognize good matches but cannot describe them precisely.

Implementation in GUI:
In the web interface, users simply click their preferred image from displayed results. The system automatically re-queries with refined features and displays updated matches. This seamless interaction requires no technical knowledge from users.

Extension 6: Web based GUI

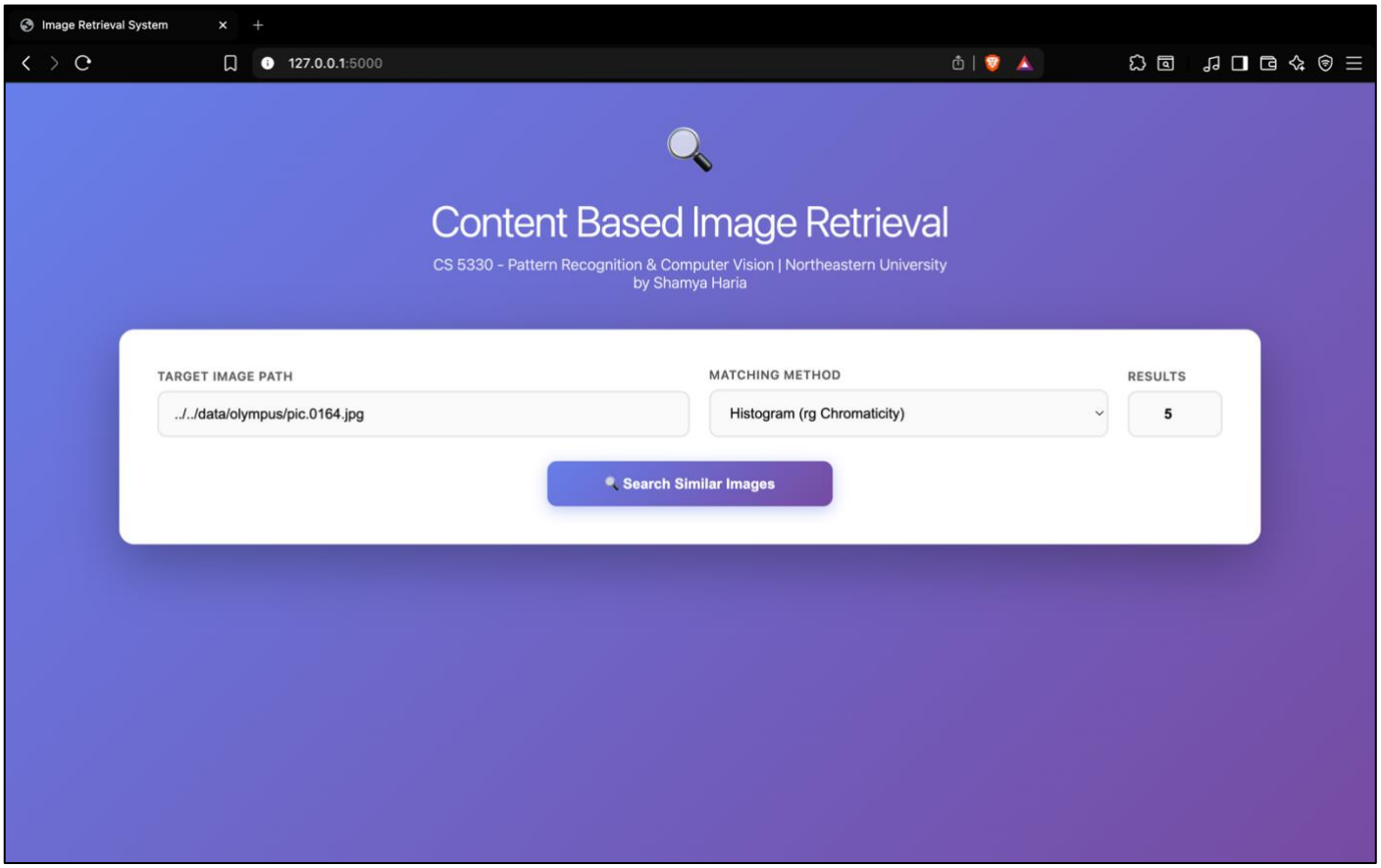

Fig 12a. Web GUI Interface
Clean interface with method dropdown and controls.

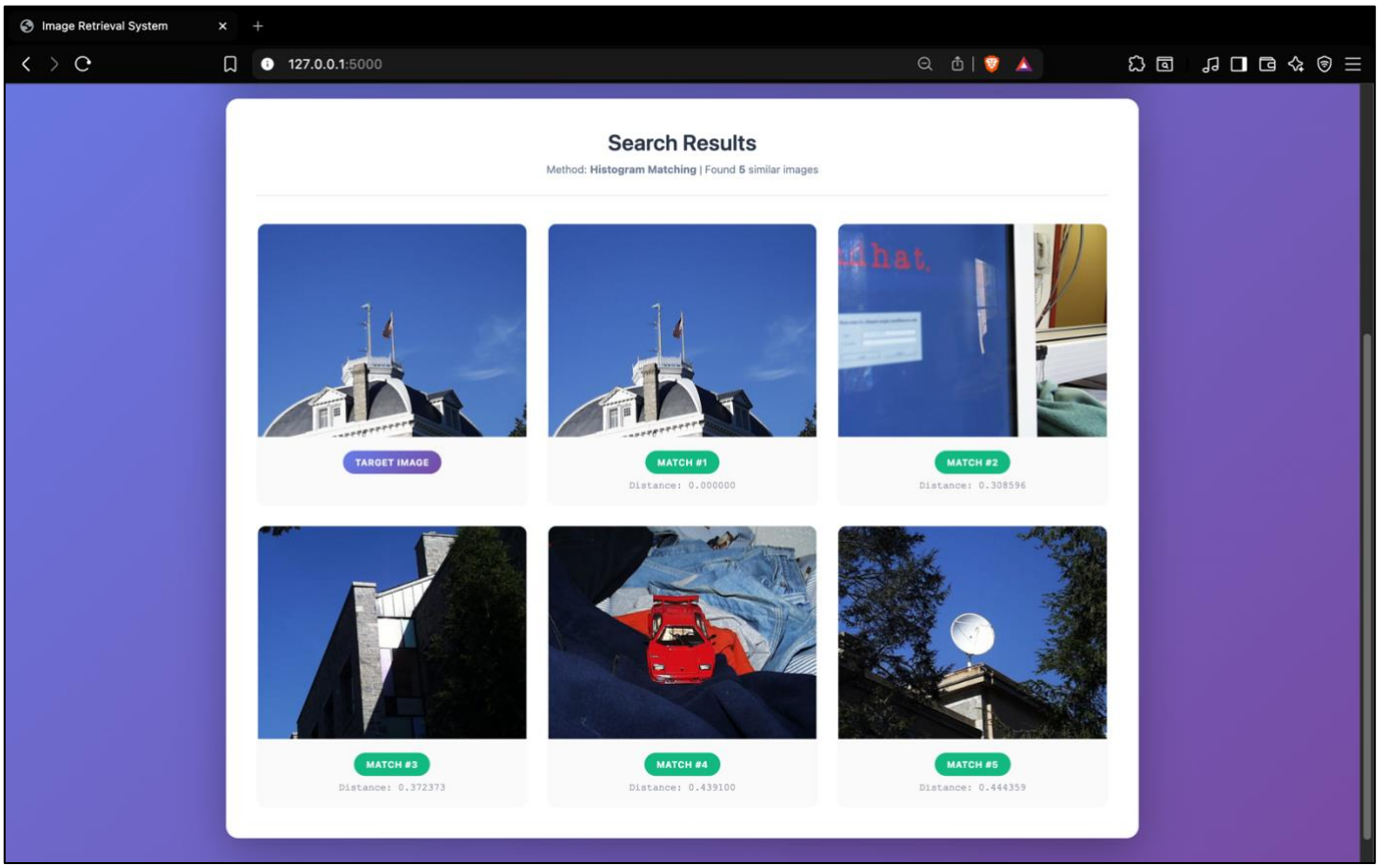

Fig 12b. GUI Results Display

Interactive grid showing matches with distance values.

While all core functionality exists as command-line tools, I developed a web-based graphical interface to make the system accessible to non-technical users. Built with Flask (Python backend) and vanilla JavaScript frontend, the GUI provides intuitive interaction with all retrieval methods.

Architecture:
The Flask server exposes REST endpoints: /search accepts POST requests with target image path, method selection, and result count, then invokes the corresponding C++ executable via subprocess and parses output. The frontend sends AJAX requests and dynamically renders results as image thumbnails in a responsive grid layout.

User Interface Features:
- Method Selection: Dropdown menu for baseline, histogram, multi-histogram, texture + color, DNN, adaptive, saliency, or query refinement
- Real-time Results: Images load progressively as the C++ backend processes queries
- Interactive Refinement: Users click any result image to trigger query refinement
- Visual Feedback: Loading indicators during computation, error messages for invalid inputs

Technical Considerations:
The subprocess approach maintains language strengths C++ for computational efficiency, Python for web framework simplicity. Images are base64 encoded for transmission, avoiding temporary file management. The stateless REST design supports multiple concurrent users without session conflicts.

Deployment:
Running 'python3 web_gui.py' starts the Flask development server on localhost:5000. Production deployment would use Gunicorn/uWSGI with nginx reverse proxy. The entire stack (C++ binaries + Python server + HTML/JS frontend) packages as a self-contained application.

REFLECTION

What I Learned:
This project deepened my understanding of how computers "see" images through quantifiable features rather than semantic interpretation. Implementing multiple feature extraction methods revealed that no single representation captures all aspects of visual similarity—color histograms excel for chromatic matching but miss texture, Sobel gradients detect edges but ignore color, and even deep learning embeddings have limitations in fine-grained color discrimination.

The most valuable lesson came from comparing method performance across diverse queries. I initially assumed DNN embeddings would dominate all classical methods, but experiments showed classical features often perform better for specific query types. Blue sky images match optimally via color histograms, textured brick walls via gradient analysis, and only semantically complex scenes truly benefit from deep embeddings. This reinforces that feature engineering remains relevant despite deep learning advances.

Implementing the adaptive weighting extension taught me how computer vision systems can exhibit "intelligent" behavior through relatively simple heuristics. Analyzing image statistics to determine appropriate feature weights requires no machine learning yet produces superior results compared to fixed combinations. This exemplifies how domain knowledge (knowing colorful images should weight color features highly) translates to algorithmic improvement.

The query refinement extension demonstrated interactive machine learning principles. Without explicitly training a model, the system learns user preferences through iterative feedback, progressively refining its understanding of desired similarity. This reinforced how AI systems need not be opaque neural networks—interpretable algorithms with human-in-the-loop feedback can achieve practical intelligence.

Challenges Overcome:
The primary technical challenge was optimizing performance for real-time interaction. Computing advanced texture features (233-dimensional vectors from Gabor/Laws/GLCM) for 1,106 images took excessive time initially. I resolved this by implementing feature pre-computation and caching, reducing query latency from minutes to seconds.

Debugging distance metrics proved subtle—incorrect normalization in histogram intersection caused meaningless distance values, and I spent hours verifying that my implementation matched theoretical formulas exactly. This experience emphasized the importance of unit testing with known-correct examples before deploying on real data.

Project Management Insights:
Completing all required tasks plus six extensions required disciplined incremental development. I systematically verified each task's correctness against provided expected results before proceeding, preventing cascading errors. The modular code structure (separate files for feature extraction, distance metrics, CSV I/O) enabled parallel development of extensions without disrupting core functionality.

Real-World Applications:
This project's techniques underpin practical systems I use daily—Google Image Search's "visually similar" results, Pinterest's visual discovery, and smartphone photo organization. Understanding implementation details demystified these "magical" technologies, revealing they combine engineered features with learned representations, just as I did in my custom design method.

Moving Forward:
Future enhancements could incorporate learned distance metrics (training a network to predict human similarity judgments), attention mechanisms weighted by detected objects (faces, text, logos), and multi-modal fusion combining visual features with metadata (timestamps, GPS locations). The modular architecture facilitates these extensions without architectural overhaul.

ACKNOWLEDGEMENTS

I acknowledge the following resources and individuals for their contributions to this project:

- Professor Bruce Maxwell for project specifications, provided image database, and ResNet18 embedding CSV file
- OpenCV library developers for comprehensive computer vision functionality
- Computer Vision by Shapiro and Stockman (Chapter 8) for theoretical foundations
- Stack Overflow community for debugging assistance with Fourier domain saliency computation

All code implementation is my original work developed **Independently**.

The project demonstrates techniques learned in CS5330 Pattern Recognition and Computer Vision at Northeastern University.